

Assignment 2: Malware Analysis

Your assignment is on Malware Analysis, which is based on the Lectures and Previous labs. The assignment is an individual assignment and is worth 30% of the module marking. You will be assessed on your ability to carry out a successful memory forensics investigation and report the artefacts and malicious activities analysed.

Marks Breakdown

You will be given a malware sample and an infected memory dump and set the task of analysing these. You are required to submit a forensics investigation report on your findings. To help you with constructing your report, Task-1 guides you to cover the main points that should be included in the report. Once you have completed the task you need to submit one report that contains the results of your investigation in PDF format.

30 Marks For all tasks Which is broken down into:

3 Marks: For clarity of your description.

27 Marks: For Task-1 question (breakdown below)

Experimental Setup

We will use the same setup of Lab 2 (Memory Forensics) to work with Volatility and Analyse the memory samples below.

You can use any instance of the Lab 2 (Memory Forensics) VM you already have or you can create a new one. To create a new one pick or create a folder. Then on a university computer right-click the folder and select "DOS shell" and then "Vagrant". If you are using your own computer then right-click and select terminal. Once you have a command prompt you can:

```
C: git clone https://git.soton.ac.uk/rht1g21/Lab2.git
```

1 Task-1

You've been given a malware sample and an infected memory dump. The malware is allegedly part of a high-scale APT attack. The antivirus industry is calling it "jackal," but detailed information is currently scarce. You searched Twitter and found someone saying "jackal's c2 list is just base64 and xor" but he didn't provide any hashes so you're not even sure if he's talking about the same executable.

```
wget https://git.soton.ac.uk/rht1g21/jackal2/-/raw/main/jackal.exe.zip unzip
jackal.exe.zip jackal.exe password: infected
wget https://git.soton.ac.uk/rht1g21/jackal2/-/raw/main/Vmem/jackal.vmem.7z sudo apt-get install
p7zip-full
7za e jackal.vmem.7z
```

1. What is the relevant profile to be used to analyse the provided memory image? (1 Mark)
A:

```
vagrant@lab2:~$ vol.py -f jackal.vmem imageinfo
Volatility Foundation Volatility Framework 2.6.1
INFO : volatility.debug : Determining profile based on KDBG search...
      Suggested Profile(s) : Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
      AS Layer1 : IA32PagedMemoryPae (Kernel AS)
      AS Layer2 : FileAddressSpace (/home/vagrant/jackal.vmem)
      PAE type : PAE
      DTB : 0x185000L
      KDBG : 0x82963c28L
      Number of Processors : 1
      Image Type (Service Pack) : 1
      KPCR for CPU 0 : 0x82964c00L
      KUSER_SHARED_DATA : 0xffdf0000L
      Image date and time : 2013-03-11 16:33:24 UTC+0000
      Image local date and time : 2013-03-11 12:33:24 -0400
```

Command Breakdown

`vol.py -f jackal.vmem imageinfo`

- `vol.py`: This command executes the Volatility framework. The output indicates that version 2.6.1 is in use.
- `-f jackal.vmem`: The `-f` parameter specifies the target file. It directs the framework to analyze the specific raw memory image, which in this instance is `jackal.vmem`.
- `imageinfo`: This specifies the Volatility plugin to be executed. Its primary function is to scan the memory dump for the Kernel Debugger Block (KDBG). This scan identifies the operating system, architecture, and patch level, which dictates the correct profile required for subsequent analysis.

The primary purpose of executing this command is to scan the raw memory image to identify the underlying operating system and determine the correct profile required for all subsequent forensic analysis.

Result Breakdown

- Suggested Profile(s) : Win7SP1x86_23418, Win7SP0x86, Win7SP1x86_24000, Win7SP1x86
- This output resolves the primary objective. The framework has identified that the target machine operated on a 32-bit (x86) version of Windows 7. Multiple options are provided because minor system updates can cause slight shifts in memory structures. Following standard practice in memory forensics, the first profile listed is typically utilized: **Win7SP1x86_23418**.

- Obtain an unpacked sample of the malware. Specifically, use procdump with and without the –memory option. Are the two output files the same? Why or why not? (4 Mark)

A:

Firstly, we will locate the malware process by running the pslist command like the image below:

```
vagrant@lab2:~$ vol.py -f jackal.vmem --profile=Win7SP1x86_23418 pslist
Volatility Foundation Volatility Framework 2.6.1
Offset(V)  Name                               PID  PPID  Thds  Hnds  Sess  Wow64  Start                               Exit
-----
0x83db4bb0 System                        4    0     91   464   -----  0  2013-03-11 15:00:13 UTC+0000
0x84640d40 smss.exe                  244  4     2    29   -----  0  2013-03-11 15:00:13 UTC+0000
0x84d3c530 csrss.exe                  336  324   9   394   0       0  2013-03-11 15:00:20 UTC+0000
0x84d63530 csrss.exe                  376  368  10   268   1       0  2013-03-11 15:00:21 UTC+0000
0x84d6d530 wininit.exe               384  324   3    75   0       0  2013-03-11 15:00:22 UTC+0000
0x85050d40 winlogon.exe              416  368   5   115   1       0  2013-03-11 15:00:22 UTC+0000
```

By scrolling through all the process, we can see the process jackal.exe.exe like the image below:

```
0x85286c20 jackal.exe.exe           3028  2340   5   162   1       0  2013-03-11 16:29:41 UTC+0000
0x852afb00 cmd.exe                  1608  3028   0   -----  1       0  2013-03-11 16:31:04 UTC+0000  2013-03-11 16
```

Look at the Parent PID (PPID) and its child processes. The jackal.exe.exe process was launched by cmd.exe (PID 2340). Even more suspiciously, the malware itself is spawning *new* cmd.exe processes (PIDs 1608 and 1084).

To see parent-child relationships, I ran the pstree command like the image below:

```
vagrant@lab2:~$ vol.py -f jackal.vmem --profile=Win7SP1x86_23418 pstree
Volatility Foundation Volatility Framework 2.6.1
Name                               Pid  PPid  Thds  Hnds  Time
-----
0x84d6d530:wininit.exe             384  324   3    75  2013-03-11 15:00:22 UTC+0000
. 0x8507e030:services.exe          480  384   7   195  2013-03-11 15:00:24 UTC+0000
.. 0x85107a58:svchost.exe           3088  480   5    72  2013-03-11 15:03:05 UTC+0000
.. 0x850e8030:vmtoolsd.exe          1412  480   9   282  2013-03-11 15:00:46 UTC+0000
.. 0x84d29098:svchost.exe            796  480  17   405  2013-03-11 15:00:33 UTC+0000
... 0x851c6b00:dwm.exe              2536  796   3    79  2013-03-11 15:02:03 UTC+0000
.. 0x852dd468:msdtc.exe             1392  480  12   143  2013-03-11 15:01:13 UTC+0000
.. 0x851b6720:taskhost.exe           1576  480   8   192  2013-03-11 15:00:56 UTC+0000
.. 0x83f528f0:svchost.exe            668  480   9   259  2013-03-11 15:00:31 UTC+0000
.. 0x85207568:svchost.exe           3116  480  14   351  2013-03-11 15:03:05 UTC+0000
.. 0x84d5b150:svchost.exe            820  480  34   891  2013-03-11 15:00:33 UTC+0000

. 0x840bd030:cmd.exe                2340  2544   1    23  2013-03-11 16:21:31 UTC+0000
.. 0x85286c20:jackal.exe.exe         3028  2340   5   162  2013-03-11 16:29:41 UTC+0000
... 0x840bb398:cmd.exe               1084  3028   1    22  2013-03-11 16:31:56 UTC+0000
... 0x852afb00:cmd.exe               1608  3028   0   -----  2013-03-11 16:31:04 UTC+0000
```

- . cmd.exe (PID 2340):** This is the parent process. It looks like whoever was on this machine (or the initial infection vector) ran a command prompt to execute the malware.
- .. jackal.exe.exe (PID 3028):** Here is our primary malware payload, running as a child of the initial command prompt.
- ... cmd.exe (PID 1084) & cmd.exe (PID 1608):** These are the most suspicious parts. The malware itself is spawning hidden command prompts!

Note: Notice how PID 1608 has ----- under the handles/threads column? That usually indicates the process has already terminated or exited in memory. Malware frequently opens a quick command shell, executes a living-off-the-land binary (like ping, whoami, or powershell), and then immediately closes it.

This completely validates that **PID 3028** is our main target. The malware is active and executing commands. Now we need to extract it from the memory dump. As a reminder, here are the two commands we need to run to get your standard dump and the memory-mapped (unpacked) dump:

```

vagrant@lab2:~$ vol.py -f jackal.vmem --profile=Win7SP1x86_23418 procdump -p 3028 --memory -D .
Volatility Foundation Volatility Framework 2.6.1
Process(V) ImageBase Name Result
-----
0x85286c20 0x00400000 jackal.exe.exe OK: executable.3028.exe

vagrant@lab2:~$ vol.py -f jackal.vmem --profile=Win7SP1x86_23418 procdump -p 3028 -D ./no_mem_option/
Volatility Foundation Volatility Framework 2.6.1
Process(V) ImageBase Name Result
-----
0x85286c20 0x00400000 jackal.exe.exe OK: executable.3028.exe
vagrant@lab2:~$ cd no_mem_option/
vagrant@lab2:~/no_mem_option$ ls
executable.3028.exe

```

To understand the need for both commands, it is first necessary to explain a defensive technique used by malware authors called packing. When malware is inactive on a hard drive, it is usually packed, meaning it is compressed or encrypted. This process hides its actual code and readable strings from antivirus detection. However, for the processor to execute the malware, the code must first decrypt and unpack itself into the system's RAM.

When the standard dump command is executed without the memory flag, Volatility examines the process in memory, reads its Portable Executable headers, and attempts to rebuild the file as if it would appear on a hard drive. Because the original file structure is reconstructed, the output represents the packed version of the malware. A strings analysis of this file mainly produces unreadable data or text related to the packing tool, while the real malicious content remains hidden.

In contrast, when the command is executed with the memory flag, Volatility bypasses the standard file headers and extracts the raw virtual memory pages where the process is running. Since the malware is actively executing in memory under PID 3028, it has already unpacked itself. As a result, this method captures the exposed payload directly from RAM. A strings analysis of this unpacked data reveals the actual instructions, hidden text.

```

vagrant@lab2:~/no_mem_option$ md5sum executable.3028.exe
22b0a433375b9ea2bb5482118df8f8de executable.3028.exe
vagrant@lab2:~/no_mem_option$ cd ..
vagrant@lab2:~$ ls
executable.3028.exe jackal.exe jackal.exe.zip jackal.vmem jackal.vmem.7z no_mem_option
vagrant@lab2:~$ md5sum executable.3028.exe
60a06e27f324572efaa4892eb692085c executable.3028.exe
vagrant@lab2:~$ |

```

By comparing the hashes of the two files, we found out that **they are not the same**. When we do a standard procdump, Volatility tries to reconstruct the original PE (Portable Executable) file using its headers, which often results in retrieving the *packed* version of the malware. However, for the malware to actually execute on the CPU, it must unpack itself in the system's RAM. By using the `--memory` flag, we are instructing Volatility to bypass the PE headers and dump the raw, active memory regions (the VAD - Virtual Address Descriptor) where the process is running. This extracts the **unpacked**, actively running payload. As a result, the hash of both files are not the same.

3. Analyse strings in the unpacked files. Make sure to use the `-a` flag to search the entire file and also check Unicode strings with `-el`. `strings -a FILENAME strings -a -el FILENAME` (3 Mark)

A:

```
vagrant@lab2:~$ strings -a -el executable.3028.exe > unicode_strings.txt
vagrant@lab2:~$ strings -a executable.3028.exe > ascii_strings.txt
```

- **From unicode_strings.txt:** We have Windows API DLLs (WUSER32.DLL), security privileges (SeDebugPrivilege), a target executable (cmd.exe), a highly suspicious block of encoded text (e.g., ISEIPSElj...), web headers (image/gif), a custom User-Agent string (...The Jackal v4.2001), registry paths, and a hardcoded file path (c:\windows\system32\jackal.exe).

- **From ascii.txt:** We have standard Windows API calls for process manipulation (CreateProcessA), memory allocation (VirtualAlloc, VirtualProtect), debugging evasion (IsDebuggerPresent), network connections (InternetConnectW, WSASocketW), and two very strange, repeating alphabet/symbol strings (!"# \$%...).

By combining these findings, we can answer the core question of what this malware does and confirm its identity:

- **Validating the APT OSINT:** The Twitter tip claimed "jackal's c2 list is just base64 and xor". We have successfully located the Base64 block in the Unicode output (notice the = and == padding at the end of lines like ISEkPSE...fw==). Furthermore, the User-Agent string explicitly says The Jackal v4.2001, confirming with 100% certainty that we are analyzing the correct sample.

- **Network Behavior (C2 Communication):** The ASCII networking APIs (WININET.dll, WS2_32.dll, URLDownloadToFileA, HttpSendRequestW) combined with the `>GET` command and the Jackal User-Agent prove this malware communicates over HTTP to download secondary payloads or receive commands from the attacker.

- **Host Infection & Persistence:** The malware drops a copy of itself at c:\windows\system32\jackal.exe. It ensures it survives reboots by modifying the Windows Registry (Software\Microsoft\Windows Player), combined with the ASCII API RegSetValueExW).

- **Evasion & Process Injection:** The presence of memory APIs (VirtualAlloc, VirtualProtect), token adjustments (OpenProcessToken, AdjustTokenPrivileges), and SeDebugPrivilege strongly indicates the malware unpacks malicious code directly into memory and injects it into legitimate processes (likely cmd.exe) to hide from antivirus. IsDebuggerPresent shows it actively tries to thwart analysts.

- **The Custom Alphabet:** Those long ASCII strings containing standard alphabets and symbols are highly abnormal. Malware authors often use custom alphabets like this to build custom Base64 decoders or generate the XOR keys needed to decrypt their payloads.

4. Based on the strings you see, describe the types of changes this malware may make to the running system's registry. Specifically, what key would you look for as an indicator of compromise? (1 Mark)
A:

```
Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; The Jackal v4.2001
Software\Microsoft\Windows Player
>GET
c:\windows\system32\jackal.exe
```

Notice how the path is software\microsoft\windows player. At first glance, it looks like a legitimate Windows component. However, the actual registry key for Windows Media Player is usually Software\Microsoft\MediaPlayer.

Malware authors frequently use a technique called "**Masquerading**." They create registry keys or processes with names that are *just* close enough to real Windows components that a system administrator glancing at the registry might ignore them. The malware is hiding in plain sight!

The Indicator of Compromise (IoC): We can state that the specific key to look for as an indicator of infection is the deceptively named **software\microsoft\windows player**.

```
vagrant@lab2:~$ grep -i "Reg" strings_output.txt
RegCloseKey
RegEnumValueW
RegQueryInfoKeyW
RegCreateKeyExW
RegSetValueExW
RegCloseKey
```

RegCreateKeyExW: This function creates a completely new registry key or opens an existing one. This is concrete proof that the malware is adding its own structure to the system's registry.

RegSetValueExW: This is the most malicious one in the list. It sets or changes the actual data stored within a registry key. This tells us the malware is actively writing information into the registry, which most likely adding a path to its own executable so that it starts up automatically when the computer boots (a technique known as persistence).

The Types of Changes: Because we found the API calls RegCreateKeyExW and RegSetValueExW, we know the malware doesn't just read the registry; it actively **creates new keys** and **modifies/writes new values** into them. It does this to establish a foothold on the system or store its configuration data.

5. Was the malware actively accessing the key at the time of the memory dump? Note: you can specifically check for this using the handles plugin and filtering for open registry keys: (Use volatility -f <FILE> handles -p PID --object-type=Key) (3 Marks)

A:

```

vagrant@lab2:~$ volatility -f jackal.vmem --profile=Win7SP1x86_23418 handles -p 3028 -t Key
Volatility Foundation Volatility Framework 2.6.1
Offset(V)  Pid  Handle  Access Type  Details
-----
0x920cf200 3028 0x18 0x20019 Key  MACHINE\SYSTEM\CONTROLSET001\CONTROL\WLS\SORTING\VERSIONS
0x8649eaf8 3028 0x20 0x20019 Key  MACHINE
0x85f7af50 3028 0x28 0x1 Key  MACHINE\SYSTEM\CONTROLSET001\CONTROL\SESSION MANAGER
0x937d3428 3028 0x78 0xf003f Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000
0x85f5a250 3028 0x7c 0xf003f Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS PLAYER
0x9366e0f8 3028 0xa8 0x2001f Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\INTERNET SETTINGS
0x91436f60 3028 0xd4 0x1 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\EXPLORER
0x85fc5f28 3028 0x130 0x20019 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\POLICIES\MICROSOFT\WINDOWS\CURRENTVERSION\INTERNET SETT
INGS
0x95b76260 3028 0x134 0x20019 Key  MACHINE\SOFTWARE\POLICIES\MICROSOFT\WINDOWS\CURRENTVERSION\INTERNET SETTINGS
0x8876e530 3028 0x138 0x20019 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\INTERNET SETTINGS
0x95afbb28 3028 0x13c 0x20019 Key  MACHINE\SOFTWARE\POLICIES
0x8da88c20 3028 0x140 0x20019 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\POLICIES
0x920e0c20 3028 0x144 0x20019 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE
0x85a34240 3028 0x148 0x20019 Key  MACHINE\SOFTWARE
0x85a1f580 3028 0x14c 0x20019 Key  MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\INTERNET SETTINGS
0x8f295030 3028 0x184 0x20019 Key  MACHINE\SYSTEM\CONTROLSET001\SERVICES\WINSOCK2\PARAMETERS\PROTOCOL_CATALOG9
0x85f7a740 3028 0x18c 0x20019 Key  MACHINE\SYSTEM\CONTROLSET001\SERVICES\WINSOCK2\PARAMETERS\NAMESPACE_CATALOGS
0x937d7350 3028 0x19c 0x20019 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\INTERNET EXPLORER\MAIN
0x9360abb8 3028 0x1a0 0x20019 Key  MACHINE\SOFTWARE\MICROSOFT\INTERNET EXPLORER\MAIN
0x85f5a7a8 3028 0x1a4 0xf003f Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000_CLASSES
0x86447730 3028 0x1b4 0x20019 Key  USER
0x921283b0 3028 0x1b8 0x3 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000
0x87980dc8 3028 0x1d8 0x20019 Key  MACHINE\SOFTWARE\MICROSOFT\TRACING\JACKAL_RASAPI32
0x95bf0030 3028 0x1f0 0x20019 Key  MACHINE\SOFTWARE\MICROSOFT\TRACING\JACKAL_RASMANICS
0x85fc2300 3028 0x248 0x20019 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\INTERNET SETTINGS\ZONE
MAP
0x865206c0 3028 0x24c 0x20019 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS NT\CURRENTVERSION\NETWORK\LOCATION AW
ARENESS
0x85a29580 3028 0x258 0x9 Key  MACHINE\SOFTWARE\MICROSOFT\WINDOWS NT\CURRENTVERSION\IMAGE FILE EXECUTION OPTIONS
0x8da179f0 3028 0x25c 0x8 Key  MACHINE\SOFTWARE\MICROSOFT\WINDOWS NT\CURRENTVERSION\APPCOMPATFLGS
0x936a4b30 3028 0x264 0x8 Key  USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS NT\CURRENTVERSION
0x95a3dc80 3028 0x268 0x20019 Key  MACHINE\SOFTWARE\MICROSOFT\WINDOWS\CURRENTVERSION\INTERNET SETTINGS\ZONEMAP
0x864a2d20 3028 0x270 0x20019 Key  MACHINE\SOFTWARE\MICROSOFT\INTERNET EXPLORER\MAIN\FEATURECONTROL\FEATURE_LOCAL_MACHINE_LOCKDOWN
0x85f5e3b8 3028 0x290 0x20019 Key  MACHINE\SOFTWARE\MICROSOFT\INTERNET EXPLORER\MAIN\FEATURECONTROL\FEATURE_ZONE_ELEVATION

```

The command used to investigate the memory dump utilizes Volatility's handles plugin to determine exactly what the malware was interacting with at the moment the system memory was captured. Operating systems grant temporary access tokens, known as handles, to processes whenever they need to open or modify an object like a file or a registry key. By specifying the malware's process ID and applying a filter to only look for registry keys, the command effectively filters out thousands of irrelevant background tasks. This allows the investigator to see a precise list of the registry keys the malware had explicitly opened and was actively holding at the exact time of the memory capture.

Looking at the terminal output generated by the command, Volatility successfully listed the registry keys the malware had open. By scanning the Details column, we can see exactly what the malware was touching. Specifically, on the fourth line of the output, associated with Handle 0x7c, the following registry path is clearly visible: USER\S-1-5-21-2833823845-3085568943-3082117713-1000\SOFTWARE\MICROSOFT\WINDOWS PLAYER. This perfectly matches the masqueraded Indicator of Compromise (IoC) we found earlier during our static string analysis. The USER\... prefix simply indicates that this key is located within the current user's hive (HKEY_CURRENT_USER).

Therefore, to answer the lab question, yes, the malware was actively accessing the key at the time of the memory dump. The presence of this specific registry path within the handle's plugin output provides definitive proof of this activity. Because the malware's process held an open handle to this exact registry key when the system's memory was frozen and captured, it confirms that the program was not just capable of modifying the registry but was caught in the active process of interacting with that specific masqueraded key.

6. Specifically, what values or data the malware add to the registry key? Is it possible using only the memory dump to find out? To query a cached registry key, use the printkey plugin like this: "volatility -f <FILE> printkey -K "Software\Microsoft\The\Key\To\Find" (3 Marks)

A:

```
vagrant@lab2:~$ volatility -f jackal.vmem --profile=Win7SP1x86_23418 printkey -K "Software\Microsoft\Windows Player"
Volatility Foundation Volatility Framework 2.6.1
Legend: (S) = Stable (V) = Volatile

-----
Registry: \??\C:\Users\Daniel\ntuser.dat
Key name: Windows Player (S)
Last updated: 2013-03-11 16:22:02 UTC+0000

Subkeys:

Values:
REG_SZ      DB1L      : (S) IionPSEjKj0rKj0nIjxgdmF6fGA8e3J/cXZhPXtnfn8=
REG_SZ      WN33      : (S) ISEqPSY9ISAgPSEjJDxndnpqajxgY39meDlyYGM=
REG_SZ      4H2N      : (S) IismPSEhKj0iJiQ9IiUrPHx9YXZwanZ7fDxldmFxcn89Z2tn
REG_SZ      MRRU      : (S) IiUrPSsnPSIqKz0hJys8YHZ2fTxqdn9jPXtnfn8=
REG_SZ      HNFY      : (S) Jic9IiAqPSIrIz0iICs8YXxyYWA8ZXJhenxmdzxyf2d2YX1yZ3p8fT17Z35/
REG_SZ      IEUH      : (S) ISYgPSEhJz0iJCI9ICo8fXZkYDxwFh5+PWN7Yw==
REG_SZ      1AUR      : (S) IisnPSscrPSInID0iIiQ8ZHJ6Z3p9dD1yYGM=
REG_SZ      47SG      : (S) ISIHPScgPSInIz0iJiE8YGN8Ywc8e3Jje3Jpcmf3PXlg
REG_SZ      FAU1      : (S) ISYhPSEhKj0iKiA9ISEkPGbnfHB4PGdhcnd6fXQ9e2d+fw==
REG_SZ      2LHL      : (S) ISAgPSIkIT0iKyM9IiUgPGFyfXB7dmE8ZHp9d3xkYD1je2M=
REG_SZ      5WYY      : (S) IicLPSshPSQkPSYqPGAqIioiIjx4f2B5ciIiPHV6f2d2YT1na2c=
REG_SZ      KYKG      : (S) IiULPSEjPSYgPSEiKjxRZmFxcn14PF9mcHp3PHLychHgPXtnfn8=
REG_SZ      Q810      : (S) IiYqPSEgIT0iIyE9IiYrPGNyYGB6fXQ8YHxmYXB2PHt8fnY9cmBj
REG_SZ      M65P      : (S) ISMhPSIqPSIqJD0iICI8fHBnfHF2YTxcgcmdmYX09eWlA=
REG_SZ      0GF9      : (S) JSQ9KyY9IScrPSEmPGRyenVgPHLmfXw9e2d+fw==
REG_SZ      IYCD      : (S) IiErPSIgJz0iJCU9IiELPHZremBnYDxDcmByd3Z9cj13fHA=
REG_SZ      780I      : (S) IiMrPSIiJj0qKj0rJTxdqn9/fGR7cn5+dmE8YXZjfGFnYA==
REG_SZ      YBTI      : (S) ICE9ISc9IScrPSIkkzx0fHx3emB7PHV2f398ZD17Z35/
REG_SZ      THRG      : (S) IiMmPSQiPSEgJD0iJTxxgZmN2YTxxwF3JxcXZhYD1rfn8=
REG_SZ      PXDT      : (S) KiM9ICY9ISAnPSEnKzx8YXp2fWc8cXJ4dmFgPXtnfn8=
REG_SZ      FYNO      : (S) IiA9KyE9IiYiPSEiJjx/fGNjdmE8d2Z+cXF2f389Z2tn
REG_SZ      IWT5      : (S) IiMiPSogPSohPSILJDx8YGd2fWB8YWo8ZHpwH3PHJ/fw==
REG_SZ      6NLE      : (S) IiQrPSsmPSIqIj0mITx2dmNhfH48YHZldmF2PXtnfn8=
REG_SZ      XOJV      : (S) ISIrPSIiID0iJCs9JCI8ZHzyZ3t2YT17Z35/
REG_SZ      XP8X      : (S) ISEkPSEhPSImJD0mPGZgcjxgfHBwdmE9e2d+fw==
REG_SZ      3EDQ      : (S) JyY9IiYiPSIrID0iJyo8eHJge3ZhPEt2YXxren10PXlg
REG_SZ      ONON      : (S) ISEiPSILJj0iJSc9JiU8Y2Z/f3FycHg8anZge3plcns9Z2tn
REG_SZ      ZXU6      : (S) ISMjPSInJz0mIz0hIiE8YXZgcn5jf3Z3PHF2dXxhdj17Z35/
REG_SZ      A3D7      : (S) ISELPSEjIj0iJCA9JCE8e3x8d2R6fXg8RXJ9cHxmZXZhPGV6YHpnPGBmfn52YT17Z35/
REG_SZ      FRRM      : (S) JyM9IiM9ISAgPSEiKzx5emBnPXlg
```

The Command: volatility -f jackal.vmem --profile=Win7SP1x86_23418 printkey -K "Software\Microsoft\Windows Player"

What the command does: This instructs Volatility to analyze the memory dump file (jackal.vmem) using a specific Windows 7 profile. The printkey plugin searches the memory structures for cached Windows Registry hives (specifically looking in RAM, not on a hard drive) and extracts the contents of the exact registry path provided (Software\Microsoft\Windows Player).

The Output: Volatility successfully finds the key inside \??\C:\Users\Daniel\ntuser.dat (the user Daniel's registry hive). It prints out a list of 29 sub-values.

Analysis: Looking at the output, we see the malware created 29 values with random 4-character alphanumeric names (like DB1L, WN33, 4H2N). The data associated with these names ends in standard padding characters like =, indicating it is Base64 encoded. However, simply decoding the Base64 yields gibberish, suggesting a second layer of obfuscation.

```

import base64

# The provided encoded data
encoded_data = """
REG_SZ DBL1 : (5) I1onPSfJk3wKJoniJsgde0f068e33/CXZhKtInfNb-
REG_SZ W033 : (5) I5eQpSV0ISAgPSEJ30vndepq8Jygv39wediYVch-
REG_SZ 4H0N : (5) I1saPSEHKJ0l1J0Q1IUkPhv0Y2Uan27H0LdaFxc08922tn
REG_SZ NRR0 : (5) I1HvPSu0n5Iqz0h3ysvY0U27f1xqd0jKZcnfna-
REG_SZ WMY : (5) J1c1t1AgpAirt081C8h00yY0A0z0hemandxyf2dYxly3p0f11735/
REG_SZ 10M : (5) I2vPSHr001T01T01C0w23Y0m0w04w0Hv7vew-
REG_SZ 1AUR : (5) I1s0P5cP5t1D08110320423y0b01Y0K-
REG_SZ 4756 : (5) I5thP5cpg6t1n201188Y0W0Vc8033j013ocw3PXlg
REG_SZ FAUL : (5) I2vPSf0k301J0515k0p0cnf0A0P6cnd0cf0Q0e2d1fw-
REG_SZ 2ULH : (5) I5AgPS1k1101K1y09110g0cyxK07dm620p0d30kY01je2H-
REG_SZ 5WVY : (5) I1c1PSH0P5QKPSYpG0G1J01Jx4f2S5c11P0W06f2d2Y71Ua2c-
REG_SZ KYGS : (5) I1H0PSf0sv0PSE1K38Z0P3cn14P0cnH30W1ych0pXtInfNb-
REG_SZ Q010 : (5) I1VpSG1g1T01Y011Y01fYvqY0B6f0Q0m0w0d2Hv0fV9c0J
REG_SZ W010 : (5) I2vPSHr001T01T01C0w23Y0m0w04w0Hv7vew-
REG_SZ 80F9 : (5) T5Q0kY0T5CfP5t0P0y0v0gH0f0eW0e2d1fw-
REG_SZ 1YCD : (5) I18rPS1g2013C0011r10W0em0eYD0cn0y010c113fMa-
REG_SZ 7001 : (5) I18rPS112J00KJ0r3J0qd0/f0R7cn5dn8Y0ZJf0r0V0-
REG_SZ V0T1 : (5) I0E9I5C0r5CfP5t0K2c0H0c0e070W02f382017235/
REG_SZ TH0G : (5) I1H0P5Q1P5G00813Tg20f02Y0u0330cX0V01rf0-
REG_SZ P00T : (5) K0M0I1C0104P5H0K20R0Q20W0C0A0m0p0XtInfNb-
REG_SZ P0V0 : (5) I10y0311V1P50131jvF0y0m0w022c0f30922tn
REG_SZ 10T5 : (5) I1H0P5c0P5H0S1J308V0G20H0S0w0d0p0w023H0JfW-
REG_SZ 00LE : (5) T1Q0P50P0T0tJ0m1T02d0W0H0W02f0d02P0KtInfNb-
REG_SZ X0TV : (5) I51rPS11D0813C003C0Z0W1y231Y71735/
REG_SZ X0BK : (5) I5K0P5H0P52nD08f0ZgcJ0gH0B0d0e2d1fw-
REG_SZ J0DQ : (5) JyV011V1P51r1D01Jy00H0ge3Z0H0T2YX0ren10PXlg
REG_SZ 00N0 : (5) I5E1P5T13J015C0J10U022f13Fy0G8B0z0c0cn8922tn
REG_SZ X000 : (5) I50P5T130m1H018V0Zgc0gJf020W020X0J1735/
REG_SZ X00Y : (5) I5E1P5T13013C0P5G0S30d0w0Sg0U030cn0Z0H0P0v0P0n0P0d0f02Y1735/
REG_SZ FR0H : (5) JyW01100150P5t0K2c0S0e0H0Xlg
...
"""

# The XOR key
xor_key = 0x00

print("Decrypted Results:")
print("-" * 50)

# Process each line
for line in encoded_data.strip().split('\n'):
    if ':(5)' in line:
        # Split the string to isolate the base64 payload
        parts = line.split(': (5) ')
        reg_name = parts[0].strip()
        b64_payload = parts[1].strip()

        try:
            # 1. Base64 Decode
            decoded_bytes = base64.b64decode(b64_payload)

            # 2. XOR Decrypt
            decrypted_bytes = bytes([b ^ xor_key for b in decoded_bytes])

            # 3. Convert bytes back to string
            decrypted_text = decrypted_bytes.decode('utf-8', errors='ignore')

            # Clean up the output to map the key name to its decrypted value
            clean_name = reg_name.split(':')[0]
            print(f'{clean_name}: {decrypted_text}')

        except Exception as e:
            print(f'Failed to decrypt {reg_name}: {e}')

```

```
Decrypted Results:
-----
DB1L      : " *!=#*+*='<`vaz`<`{r qva={g~
WN33      : !|*=&!    =!$<gvzjj<`c fx=r`c
4H2N      : "+&=!|*=&$~*%+<|}avpjv{<|evaqr =gkg
MRRR      : "%&+="+*+!="<`+<`vv`<jv c={g~
HNFY      : &`="  *="+#="  +a|ra`keraz|fwkr gva}rgz|}={g~
IEUH      : |& =!|`"=$~" *%<`vd`<p|nrv=C{c
1AUR      : "+`='+'+"="=" "$<drzgz`t=r`c
475G      : !`!`="  "="#&|`<`c|ag<{rc{r!raw=y`
FAU1      : |&|!=!|*="*  =!|!$<`g|px<garwz`<t={g~
2LHL      : !  = "$!="+#=" "% <ar`p{va<dzjw|d`<`c{c
5WYY      : ""%+=!=$&=&*<`*""*`<x `yn`""<uz gva=gkg
KYKG      : "%%<#=&=&  =!`*`<Qfaqr`x<`fpzkyrpx`={g~
Q810      : "`&*`=!  !`="#!`="&+<cr`z`<`<`fapv<{|~v=r`c
M65P      : !#`!`="*="*$~"  "<|pg|qva<`rgfa`<y`
0GF9      : %$+=&`!`+'!&<drzu`<cyf`|={g~
IYCD      : "!`!`="  "=$~"!="&vkvz`g`<Cr`rvw`<r=w|p
78OI      : "#+=""&*`="+=<`jv  |d{r~vavacv|ag`
YBTI      : !`!`!`!`+'="+<`t|!wz`<`<uv  |d={g~
THRG      : "#&=$`=!  $~"%"<`fcva<p rqqva`<k~
PXD0      : *#`=  &`!  !`+'<|azv`g<qrxva`={g~
FYN0      : "  =+!`="&`="!<`<ccva<wfv~qqv  =gkg
IWT5      : "#`=" *  *`!="%$<|`gv`>`aj<dzpxvv<r<
6NLE      : "$+=+&="*`="&|<vvca|~<`<vevav={g~
...
ONON      : !|`!`="&`="%"`<=&%<cf qrp<`jv`>{zer={gkg
ZXU6      : |#`="""`<=&#`="!<`av`r~c vvkquv|av={g~
A307      : !|!`!`#`="!`$  =!<{|!|wdz`<x{Er`p|feva<ez`zg<`f~vva={g~
FRRM      : `#`="#`!  *`="!`+<yz`<g~y`

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

output, a distinct pattern emerged: groups of characters separated by an equals sign (=), perfectly mirroring the structure of an IPv4 address (e.g., XXX=XXX=XXX=XXX). Deducing that the = character in the ciphertext was actually a substitute for the . (dot) in a standard IP address, a known-plaintext calculation could be applied. The ASCII hex value for = is 0x3D, and the hex value for . is 0x2E. XORing the ciphertext byte with the known plaintext byte ($0x3D \oplus 0x2E$) reveals the hidden key: 0x13. Applying this 0x13 XOR key to the entirety of the data successfully decrypted the payloads.

```
import base64

# The provided encoded data
encoded_data = """
R00_Z2 DH1L 1 (5) 110P5EjHjarc3m1jgde4f6A6e3j/cz0Pntf0f0-
R00_Z2 MD3 1 (5) 15Gp0V5154gP5f3W0ndepq1qg7f0e6tVYp-
R00_Z2 402N 1 (5) 11a0P5Hjg11q0t110P450V2Za0z7f0d3d0cc0022ta
R00_Z2 H0RU 1 (5) 110P5EjP542C0W05Y0Y02f1q0d0j0Y0f0f0-
R00_Z2 H0RY 1 (5) 11c0T1aP5Tr10t10T040Y0Y0M0C0h0e0nd0ay024P00Y07j0hT1X7Z5/
R00_Z2 10uR 1 (5) 110P5Ej0110T10C0C0X00f00f0e0nd0ay0-
R00_Z2 1A0R 1 (5) 110P5EjP5410W110C02020j0d0Y0Y0f0-
R00_Z2 4Y56 1 (5) 115H05EjP5T0e10T10Y0Y0W0e0e3j0j0u0f0P01g
R00_Z2 F0U1 1 (5) 115H05EjKj01C0Y150K0P0H000P0d0e0nd0f0q0e0d0f0-
R00_Z2 2U0L 1 (5) 115Gp0T0C0T0Y0M110gP0Y0K0Z0d0030d0e0k0Y01e0P0-
R00_Z2 50Y9 1 (5) 11c0P0P50P50Y0P0e0j10t110j0P0C011P0M020Y0T10d0-
R00_Z2 K0W0 1 (5) 1101P5EjP0q0P50K0j0e20f0c010P0e0c0P0Y0c0m0g0t0f0-
R00_Z2 0Q0S 1 (5) 11V0P5EjT0T10Y01V0P0Y0Y0d0E10Q0A0V0Y02P0H0f0e0c0j
R00_Z2 N05P 1 (5) 115H05EjP50g100T10T0f0H0f0C0Y0g0u0f0W0u0-
R00_Z2 0G09 1 (5) 115Gp0Y0Y0C0P50E0Y0g0Y0T0f0A0d0e0d0f0-
R00_Z2 T0YD 1 (5) 11E0P5T0g001C0W0T0E10P0C0e0d0f0d0e0k0Y020j10f0-
R00_Z2 T0C1 1 (5) 110P5E11j0g0d0r10d0f0f0k10c0h0e0nd0f0R0f0e0-
R00_Z2 Y0H1 1 (5) 115G05E0505E0100K0e0f00d0P0M0Z0P0e0d0Z0Z0/
R00_Z2 70B0 1 (5) 110P5E05P5G000110g20M0Y0T0f030C0Y0Y0T0f0-
R00_Z2 P0M0 1 (5) 110H0Y0T150u0P50E0e0c0Y0P0f0C0C0X040e0P0C0f0-
R00_Z2 F0M0 1 (5) 1100Y0E01V0P5010Y0Y050E080Z0C0Z0f08090Z0-
R00_Z2 20T5 1 (5) 1101P0e0P0e0P5110b0Y0C0Z0f0M0A0T0e0e020M0j0-
R00_Z2 00E1 1 (5) 11Q0P50E05T0130e0T0Z0d0e0M0000210M0Z000f0-
R00_Z2 X0Z9 1 (5) 1150P501100010C050C0E0H0Y0Y020Y11Z10/
R00_Z2 X0X0 1 (5) 115E0P5P05010000G0Z0j0g0H0d0E0e0d0f0-
R00_Z2 30Q0 1 (5) 11Y0V1Y50t0e0H0j0a0e0G0j0P0E10C00000000K0g
R00_Z2 00K0 1 (5) 115G1P511301P50Y010W0Z0f0Y0c0h0e0Z0e0C000Z0-
R00_Z2 20P0 1 (5) 110P5Ej010b0d0f0f0d0f0g050T0E10H0P0e0j10Y0Z0/
R00_Z2 A007 1 (5) 1101P50110C0X0e000000P0g0000C0e0d0Z0P0e0P0e0f00f0Y0T11Z15/
R00_Z2 F0R0 1 (5) 11Y0H1P0150P5L0K0C0e0M0K0g
"""

# The XOR key
xor_key = 0x15

print("Decrypted Results:")
print("-" * 60)

# Process each line
for line in encoded_data.strip().split('\n'):
    if "(5)" in line:
        # Split the string to isolate the Base64 payload
        parts = line.split(" (5) ")
        reg_name = parts[0].strip()
        b64_payload = parts[1].strip()

        try:
            # 1. Base64 Decode
            decoded_bytes = base64.b64decode(b64_payload)

            # 2. XOR Decrypt
            decrypted_bytes = bytes([b ^ xor_key for b in decoded_bytes])

            # 3. Convert Bytes back to string
            decrypted_text = decrypted_bytes.decode('utf-8', errors='ignore')

            # Clean up the output to map the key name to its decrypted value
            clean_name = reg_name.split()[1]
            print(f"({clean_name}:{5}) : {decrypted_text}")

        except (UnicodeError as e):
            print(f"Failed to decrypt {reg_name}: {e}")
```

```
Decrypted Results:
-----
DB1L      : 194.209.89.41/serios/halber.html
WN33      : 229.5.233.207/teiy/spluk.asp
4H2N      : 185.229.157.168/onrecyeho/verbal.txt
MRRU      : 168.84.198.248/seen/yelp.html
HNFY      : 54.139.180.138/roars/varioud/alternation.html
IEUH      : 253.224.171.39/news/comm.php
1AUR      : 184.48.143.117/waiting.asp
47SG      : 212.43.140.152/sport/haphazard.js
FAU1      : 252.229.193.227/stock/trading.html
2LHL      : 233.172.180.163/rancher/windows.php
5WYY      : 146.82.77.59/s91911/klsja11/filter.txt
KYKG      : 166.20.53.219/Burbank/Lucid/jacks.html
Q8I0      : 159.232.102.158/passing/source/home.asp
M65P      : 202.19.197.131/october/saturn.js
0GF9      : 67.85.248.25/waifs/juno.html
IYCD      : 128.134.176.126/exists/Pasadena.doc
780I      : 108.115.99.86/yellowhammer/reports
YBTI      : 32.24.248.178/goodish/fellow.html
THRG      : 105.71.237.16/super/clabbers.xml
PXDT      : 90.35.234.248/orient/bakers.html
FYNO      : 13.82.151.215/lopper/dumbbell.txt
IWT5      : 101.93.92.167/ostensory/wicked/all
6NLE      : 178.85.191.52/eprom/severe.html
...
ONON      : 221.165.164.56/pullback/yeshivah.txt
ZXU6      : 200.144.50.212/resampled/before.html
A3D7      : 226.201.173.72/hoodwink/Vancouver/visit/summer.html
FRRM      : 40.10.239.218/jist.js

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Now, using the analysis from the steps above, here are the direct answers to the initial questions:

1. Specifically, what values or data did the malware add to the registry key?

The malware added **30 specific registry values** to the HKCU\Software\Microsoft\Windows Player key.

The Values: It generated random 4-character alphanumeric names (e.g., DB1L, WN33, 47SG).

The Data: It added Base64-encoded and XOR-obfuscated strings. Once decrypted, this data was revealed to be a list of malicious IP addresses, URLs, and file paths that serve as the malware's Command and Control (C2) infrastructure.

2. Is it possible using only the memory dump to find out?

I don't really understand the question, so I divide into 2 cases.

Case 1: If the question means, "Can we read the actual payload values just by dumping the registry key?"

The answer is no. Simply dumping the registry values using the printkey command only provides the obfuscated strings. To find out what data the malware actually added, we must manually decode the Base64 and apply the XOR decryption (using the 0x13 key) to reveal the real Command and Control (C2) URLs and IP addresses.

Case 2: If the question means, "Can we gather all the necessary evidence to find this out using *only* the memory dump file (without needing the physical hard drive)?"

The answer is yes, it is entirely possible using only the memory dump. When a Windows operating system runs, it caches actively used portions of the Windows Registry directly into RAM to improve system performance. Volatility's printkey plugin doesn't need a hard drive; it scans the memory dump (jackal.vmem) to locate these cached CMHIVE registry structures in RAM and rebuilds the key exactly as it existed at the time the machine was captured. Furthermore, if we hadn't deduced the 0x13 key through known-plaintext analysis, we could have used Volatility to extract the actual malware

executable from memory (using plugins like procdump) and reverse-engineered it to find the decryption routine.

7. Are there any specific network artefacts that you would configure an Intrusion Detection System to look for? (2 Marks)

A:

1. Destination IP Addresses (Blacklisting)

The most immediate artifact is the IP address. We would configure the IDS to trigger a high-priority alert (or configure a firewall to block) any outbound network connection originating from our internal network destined for any of the 30 IP addresses on that list.

IP addresses to flag:

194.209.89.41
229.5.233.207
185.229.157.168
168.84.198.248
54.139.180.138
253.224.171.39
184.48.143.117
212.43.140.152
252.229.193.227
233.172.180.163
146.82.77.59
166.20.53.219
159.232.102.158
202.19.197.131
67.85.248.25
128.134.176.126
108.115.99.86
32.24.248.178
105.71.237.16
90.35.234.248
13.82.151.215
101.93.92.167
178.85.191.52
218.113.178.71
227.22.157.5
45.151.183.149
221.165.164.56
200.144.50.212
226.201.173.72
40.10.239.218

IDS Logic: "Alert if Source IP is [Internal Subnet] and Destination IP is [Malicious IP List]."

2. Specific HTTP URI Paths

Malware often uses highly specific, hardcoded directory structures on their Command and Control (C2) servers. An IDS can inspect the HTTP traffic (specifically the GET or POST requests) and look for these unique folder paths in the URI.

Examples to flag: /series/, /roars/varioud/, /s91911/klsja11/, /hoodwink/Vancouver/visit/.

IDS Logic: "Alert if HTTP Request URI contains '/s91911/klsja11/'."

3. Exact File Names and Extensions

The malware is reaching out to download specific files. The IDS can be configured to alert if it sees HTTP requests asking for these exact filenames or hashes, or if it sees network traffic delivering files with these names.

Examples to flag: halber.html, spluk.asp, Xeroxing.js, Pasadena.doc.

IDS Logic: "Alert if HTTP Request URI contains 'Xeroxing.js' or 'spluk.asp'."

4. Suspicious Combinations (High-Fidelity Rules)

To avoid false positives (for example, triggering an alert just because someone legitimately downloaded a file called home.asp), an IDS is best configured by combining these artifacts into a single, highly specific rule.

Example Rule: "Alert on TCP traffic over Port 80 WHERE the Destination IP is 146.82.77.59 AND the HTTP GET request is asking for /s91911/klsja11/filter.txt."

8. What's the name of the mutex this malware uses? (3 Marks)

A:

```
vagrant@lab2:~$ vol.py -f jackal.vmem --profile=Win7SP1x86_23418 handles -p 3028 -t mutant
Volatility Foundation Volatility Framework 2.6.1
```

Offset(V)	Pid	Handle	Access	Type	Details
0x85292cd8	3028	0x1c	0x1f0001	Mutant	
0x84d2ad20	3028	0x6c	0x1f0001	Mutant	__Dassara__
0x8517e330	3028	0xf8	0x1f0001	Mutant	__!MSFTHISTORY!__
0x84d4de18	3028	0xfc	0x1f0001	Mutant	c:\users\daniel\appdata\local\microsoft\windows\temporary internet files\content.ie5!
0x85293118	3028	0x108	0x1f0001	Mutant	c:\users\daniel\appdata\roaming\microsoft\windows\cookies!
0x8532ffe0	3028	0x114	0x1f0001	Mutant	c:\users\daniel\appdata\local\microsoft\windows\history\history.ie5!
0x840ccdb8	3028	0x158	0x1f0001	Mutant	WininetStartupMutex
0x85428700	3028	0x1a8	0x1f0001	Mutant	WininetConnectionMutex
0x84d76fe0	3028	0x1ac	0x1f0001	Mutant	WininetProxyRegistryMutex
0x85272368	3028	0x1bc	0x1f0001	Mutant	RasPbFile
0x852592a0	3028	0x274	0x1f0001	Mutant	ZonesCacheCounterMutex
0x851ee850	3028	0x27c	0x1f0001	Mutant	ZonesLockedCacheCounterMutex

Based on the Volatility output provided, the name of the mutex used by the malware is __Dassara__. Here is a quick breakdown of how to spot it:

- __Dassara__ is a highly suspicious, non-standard string. Malware often creates a uniquely named mutex like this to act as an infection marker, ensuring that only one instance of the malicious payload runs on the system at a time.
 - **The rest of the list** consists of standard, legitimate Windows and Internet Explorer mutexes. For example, WininetStartupMutex, ZonesCacheCounterMutex, and the strings containing file paths (c:\users\daniel...) are normal artifacts of standard OS operations.
9. List as many IPs/hostnames used by the malware. Is the guy right about it using base64 and xor? (3 Marks)

A:

IP addresses:

194.209.89.41
229.5.233.207
185.229.157.168
168.84.198.248
54.139.180.138
253.224.171.39
184.48.143.117
212.43.140.152
252.229.193.227
233.172.180.163
146.82.77.59
166.20.53.219
159.232.102.158
202.19.197.131
67.85.248.25
128.134.176.126
108.115.99.86
32.24.248.178
105.71.237.16
90.35.234.248
13.82.151.215
101.93.92.167
178.85.191.52
218.113.178.71
227.22.157.5
45.151.183.149
221.165.164.56
200.144.50.212
226.201.173.72
40.10.239.218

Is the guy right about it using base64 and xor?

Yes, the OSINT researcher on Twitter was absolutely right. The initial raw strings contained = and == padding, which is a textbook indicator of Base64 encoding.

10. Can you detect network activity from the malware and if so which network protocol is it using, on which local port and what is it doing (what is the state)? (4 Marks)

A:

```
vagrant@lab2:~$ volatility -f jackal.vmem --profile=Win7SP1x86_23418 netscan
Volatility Foundation Volatility Framework 2.6.1
Offset(P)      Proto  Local Address      Foreign Address    State      Pid      Owner      Created
0x1e93c570     UDPv4  0.0.0.0:5355       *:.*               *          1064     svchost.exe 2013-03-11 16:22:02 UTC+0000
0x1e93c570     UDPv6  ::5355            *:.*               *          1064     svchost.exe 2013-03-11 16:22:02 UTC+0000
0x1e802838     TCPv4  0.0.0.0:49156      0.0.0.0:0         LISTENING  480     services.exe
0x1e826508     TCPv4  172.16.237.134:9090 172.16.237.1:50645 ESTABLISHED -1
0x1e92b008     TCPv4  172.16.237.134:9090 172.16.237.1:50606 CLOSE_WAIT -1
0x1ef6b740     UDPv4  172.16.237.134:138 *:.*              4       System      2013-03-11 15:00:22 UTC+0000
0x1ef6d450     UDPv4  172.16.237.134:137 *:.*              4       System      2013-03-11 15:00:22 UTC+0000
0x1ea34940     TCPv4  172.16.237.134:139 0.0.0.0:0         LISTENING  4       System
0x1eac1008     TCPv4  0.0.0.0:135        0.0.0.0:0         LISTENING  668     svchost.exe
0x1eac1008     TCPv6  :::135            :::0               668     svchost.exe
0x1eac2ca8     TCPv4  0.0.0.0:135        0.0.0.0:0         LISTENING  668     svchost.exe
0x1eaf60d8     TCPv4  0.0.0.0:49155      0.0.0.0:0         LISTENING  820     svchost.exe
0x1eb338c8     TCPv4  0.0.0.0:9090       0.0.0.0:0         LISTENING  3028    jackal.exe.exe
0x1ebfa140     TCPv4  0.0.0.0:445        0.0.0.0:0         LISTENING  4       System
0x1ebfa140     TCPv6  :::445            :::0               4       System
0x1ebfc490     TCPv4  0.0.0.0:49156      0.0.0.0:0         LISTENING  480     services.exe
0x1ebfc490     TCPv6  :::49156          :::0               480     services.exe
0x1ef4b370     TCPv4  0.0.0.0:49153      0.0.0.0:0         LISTENING  756     svchost.exe
0x1ef4b370     TCPv6  :::49153          :::0               756     svchost.exe
0x1ef569f0     TCPv4  0.0.0.0:49154      0.0.0.0:0         LISTENING  488     lsass.exe
0x1ef569f0     TCPv6  :::49154          :::0               488     lsass.exe
0x1ef64c78     TCPv4  0.0.0.0:49154      0.0.0.0:0         LISTENING  488     lsass.exe
0x1fa91520     UDPv4  0.0.0.0:0          *:.*               1064     svchost.exe 2013-03-11 16:22:02 UTC+0000
0x1fa91520     UDPv6  :::0              *:.*               1064     svchost.exe 2013-03-11 16:22:02 UTC+0000
0x1faa1200     UDPv4  0.0.0.0:5355       *:.*               1064     svchost.exe 2013-03-11 16:22:02 UTC+0000
0x1faef990     UDPv6  fe80::bdf:f616:7b59:1e75:546 *:.*               756     svchost.exe 2013-03-11 16:33:11 UTC+0000
0x1f7c3378     TCPv4  0.0.0.0:49153      0.0.0.0:0         LISTENING  756     svchost.exe
0x1f7c3378     TCPv6  0.0.0.0:49153      0.0.0.0:0         LISTENING  384     wininit.exe
0x1f7c9150     TCPv4  0.0.0.0:49152      0.0.0.0:0         LISTENING  384     wininit.exe
0x1f7c9150     TCPv6  :::49152          :::0               384     wininit.exe
0x1f7ca2a8     TCPv4  0.0.0.0:49152      0.0.0.0:0         LISTENING  384     wininit.exe
0x1fa09f60     TCPv4  0.0.0.0:49155      0.0.0.0:0         LISTENING  820     svchost.exe
0x1fa09f60     TCPv6  :::49155          :::0               820     svchost.exe
0x1fb17b00     TCPv6  fe80::bdf:f616:7b59:1e75:49168 ff02::1:2:445     CLOSED     -1
0x1fd8f008     TCPv6  fe80::bdf:f616:7b59:1e75:445  ff02::1:2:49168  CLOSED     -1
```

Based on the netscan output provided, we can definitively detect active network behavior originating from the malware. By locating process ID (PID) 3028, which was previously identified as the malicious executable jackal.exe.exe, we can see its network footprint in memory. The scan reveals that this process is explicitly configured to use the TCP protocol over IPv4.

Looking at the specific socket details, the malware has bound itself to local port 9090 on the infected machine. Crucially, the current state of this connection is marked as LISTENING. This indicates that the malware has opened a local port and is actively standing by to receive incoming network connections, which is standard behavior for a backdoor or a bind shell waiting for remote instructions.

Furthermore, a comprehensive analysis of the scan reveals additional context regarding this port. Although the process ID is abstracted (listed as -1 for terminated threads or OS-level socket handling), there are two other entries operating on that exact same local port of 9090. These entries show connections with an external IP address (172.16.237.1) in both ESTABLISHED and CLOSE_WAIT states. This proves that not only was the malware listening, but an external entity successfully connected to it just before or during the time the memory capture was taken.